INTRO TO LLM & TRANSFORMER

Topics on NLP & LLM
- ADDI Laboratory -

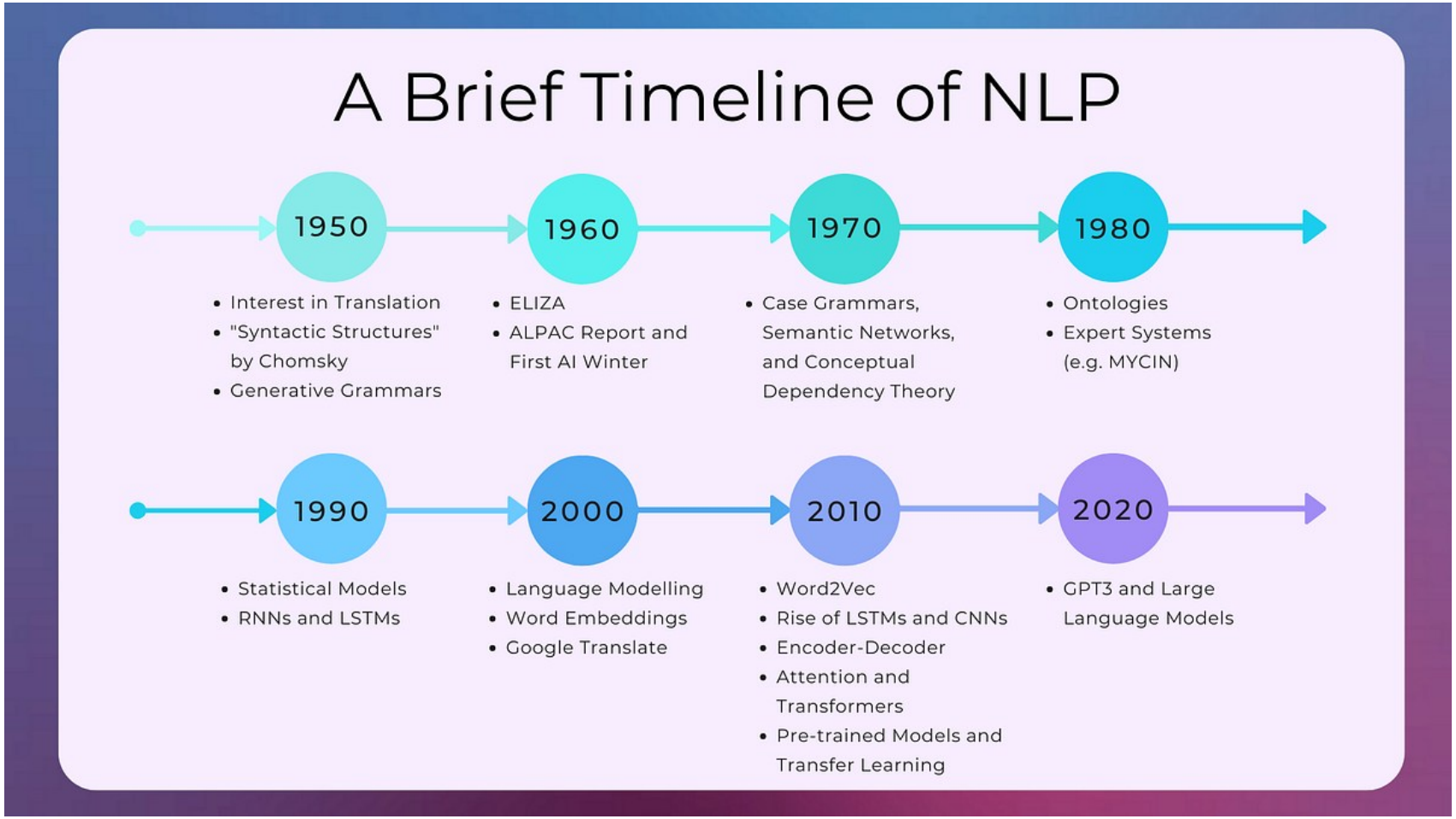
Rarasmaya Indraswari
(WA) 081331818454
raras@its.ac.id

Contents

- NLP evolution
- From Words to Vector
- RNN vs Transformer
- Transformer to LLM

NLP Evolution

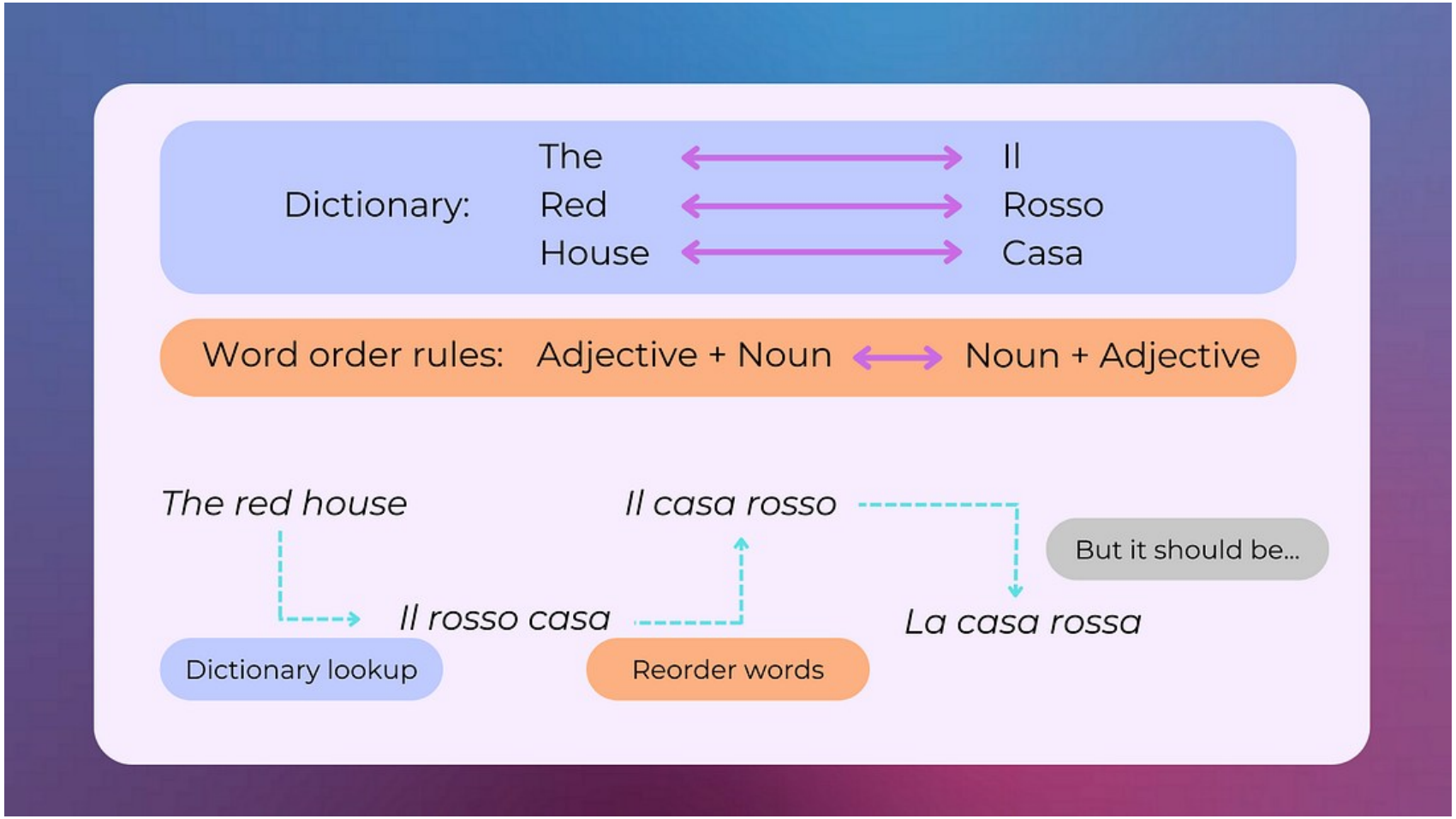
History of NLP



The 1950s, 1960s, and 1970s: Hype and the First AI Winter

- The 1950s saw a lot of excitement around the potential for fully automatic high-quality translation systems.
- However, this was quickly dampened by the reality that such systems were far from achievable at the time.
- These systems were not very effective, as they did not take into account the ambiguity of natural language.
- In 1966, the ALPAC (Automatic Language Processing Advisory Committee, a committee established in 1964 by the United States government in order to evaluate the progress in computational linguistics) released a report that recommended research into machine translation be halted, which had a significant impact on research in natural language processing and artificial intelligence more broadly.

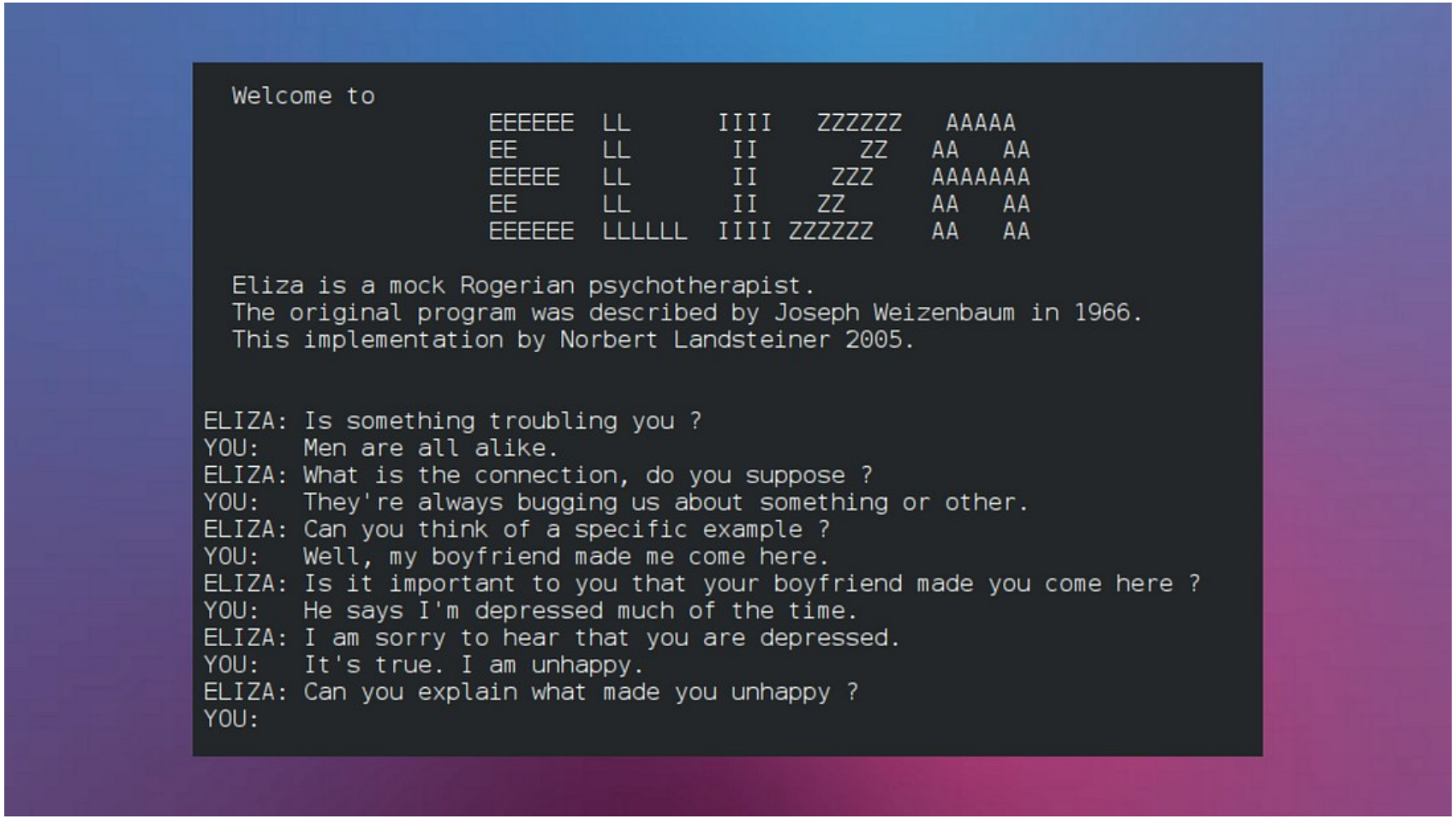
The 1950s, 1960s, and 1970s: Hype and the First AI Winter



The 1950s, 1960s, and 1970s: Hype and the First AI Winter

- In addition to the development of language theories, several prototypes were also created during this period.
- The most famous prototype is perhaps ELIZA, designed to replicate the conversation between a psychologist and a patient.
- Despite the slowdown in NLP research in the 1970s, there have been developments in new computationally tractable theories of grammar, such as case grammars, semantic networks, and conceptual dependency theory.

The 1950s, 1960s, and 1970s: Hype and the First AI Winter



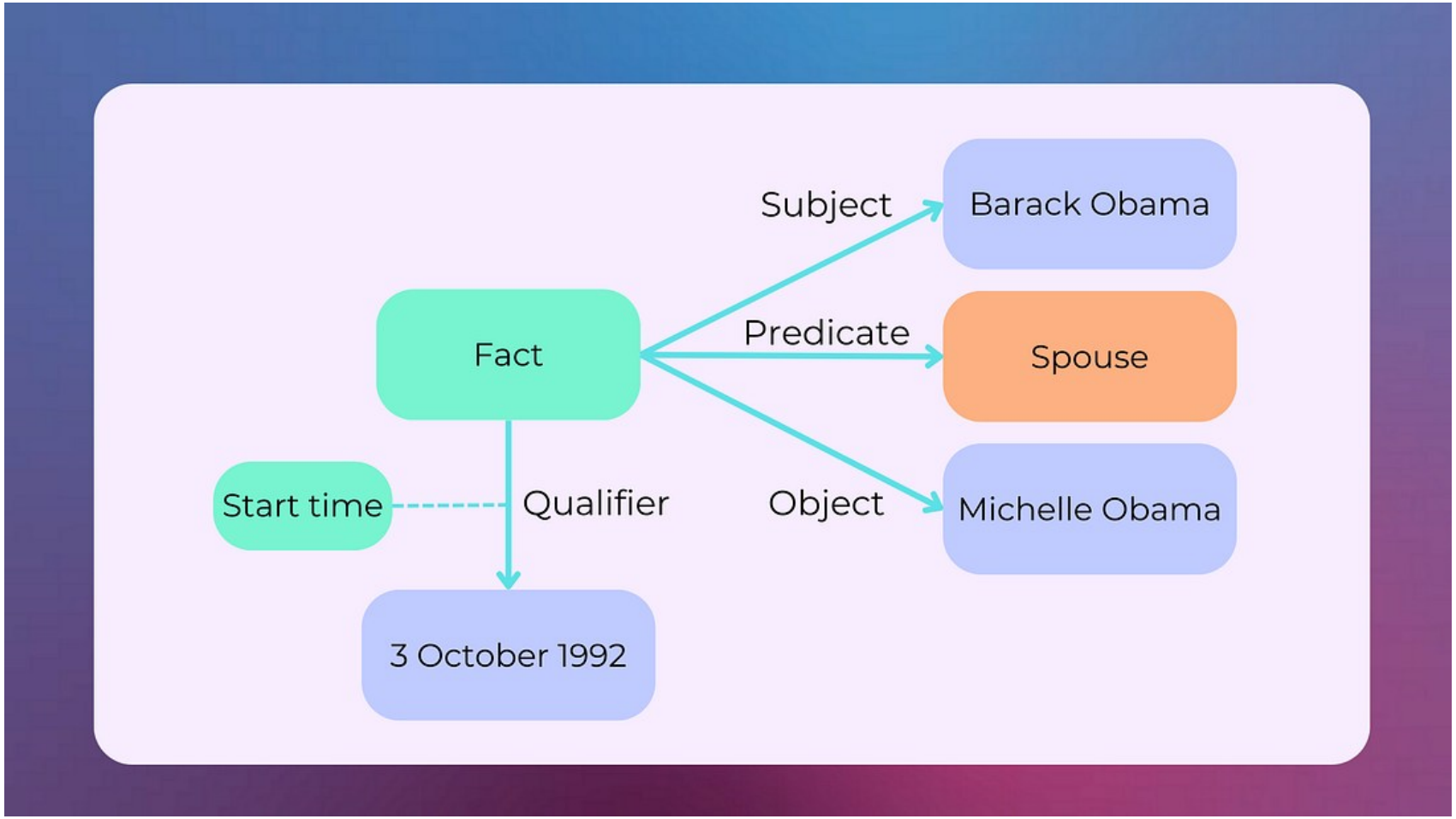
The 1950s, 1960s, and 1970s: Hype and the First AI Winter

- Early NLP systems relied heavily on human-designed linguistic rules (if-then rules).
- For example, a program that recognized weather queries ("If a sentence contains the word 'weather', then classify it as a weather query").
- The main problems were: they were susceptible to language variations, required significant maintenance, and were difficult to scale.

The 1980s and 1990s: Expert Systems and Statistical Models

- In the 1980s, symbolic approaches (also called expert systems) were used in NLP, with hard-coded rules and ontologies (i.e. a knowledge base of facts, concepts, and their relationships about a specific domain).
- These were programs designed to mimic human experts in specific domains, such as medicine or law. One of the most famous expert systems was MYCIN, which was used to diagnose blood infections.
- In the late 1980s and early 1990s, statistical models took over the symbolic approach.
- Statistical models were able to learn by themselves, through machine learning, the multitude of hard-coded rules of the expert systems.
- This was made possible above all by the increase in computational resources.
- The first recurrent neural networks (RNNs) were trained in these years.

The 1980s and 1990s: Expert Systems and Statistical Models



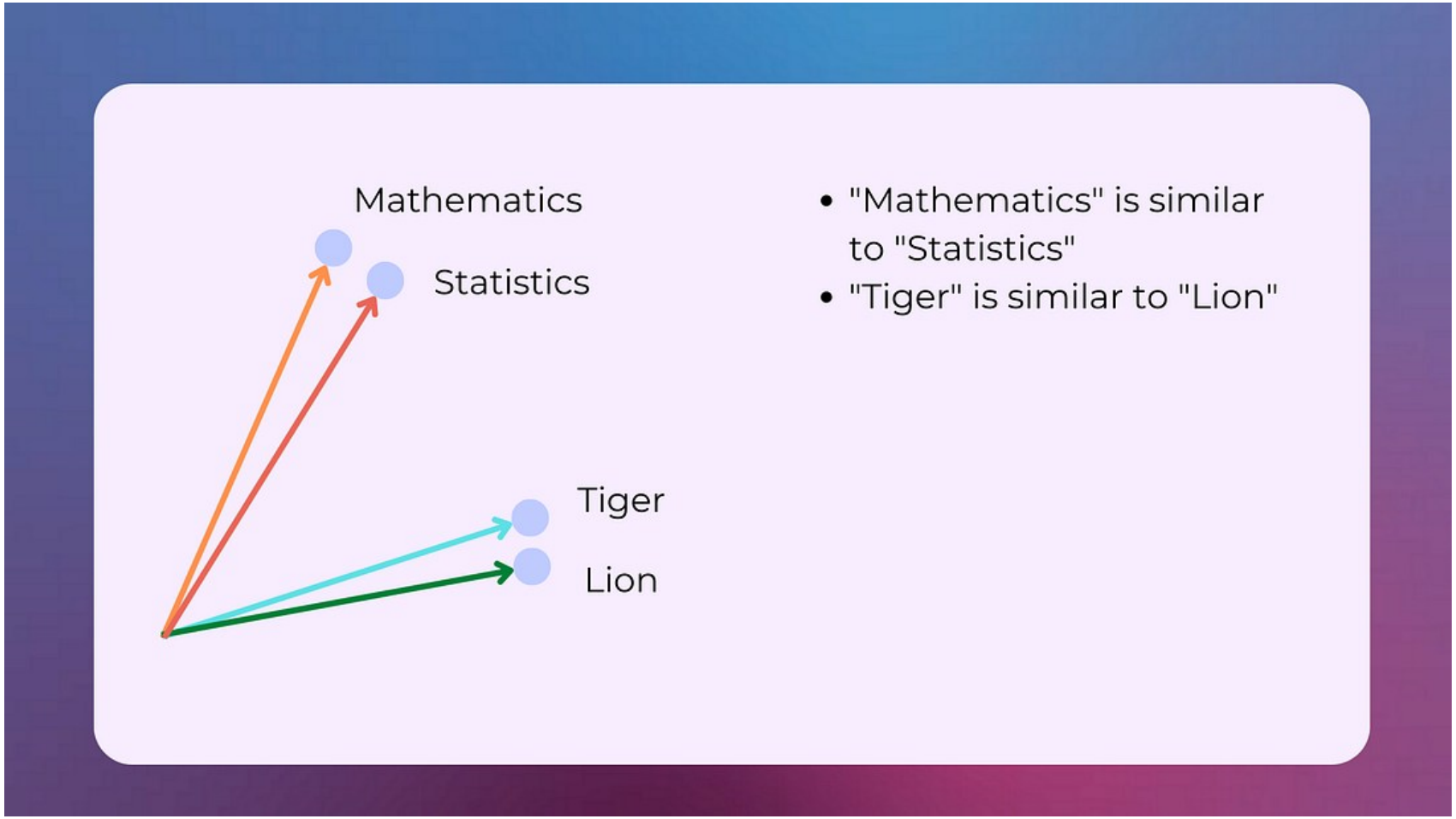
The 1980s and 1990s: Expert Systems and Statistical Models

- With the advent of large corpora and improved computing, probabilistic methods like n-grams, HMMs, and CRFs have begun to be applied.
- These systems learn from data to model language, not just human rules.
- Advantages: more flexible, scalable than rule-based methods.
- Limitations: still dependent on n-gram length, limited contextualization.

The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers

- In the 2000s, neural networks are used more and more, initially for learning the distribution of language words and predicting the next words in a text given the previous ones (i.e. language modeling).
- Also in this period, we begin to represent words with dense vectors of numbers called word embedding, so that words with similar meanings are associated with similar vectors.
- However, early algorithms were unable to efficiently learn this representation and so they were trained on a small amount of text, thus producing suboptimal vectors.
- The first commercially successful natural language processing system was the Google Translate service, which was launched in 2006.
- Google Translate used statistical models to automatically translate documents from one language to another.

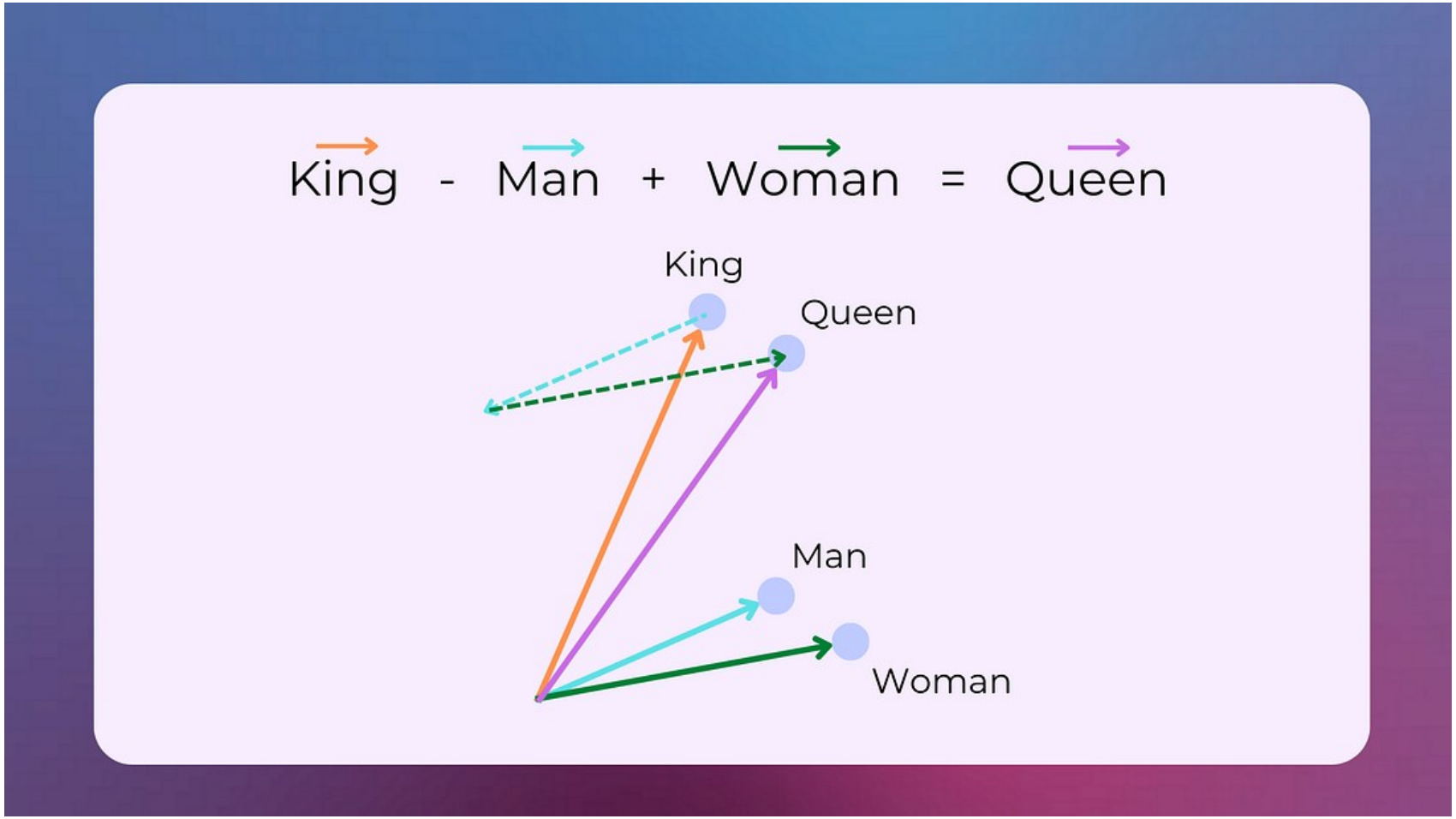
The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers



The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers

- In 2013 the Word2Vec paper "Efficient Estimation of Word Representations in Vector Space" was published, i.e. the first algorithm capable of learning word embedding efficiently, thus greatly improving previous word embedding.
- In the first experiments with these vectors, it was noticed that it is possible to do mathematical operations with them that represent semantic operations.
- For example, taking the vector of the word "king", subtracting the vector of "man" from it and adding the vector of "woman", we get a vector very close to the vector of "queen".

The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers



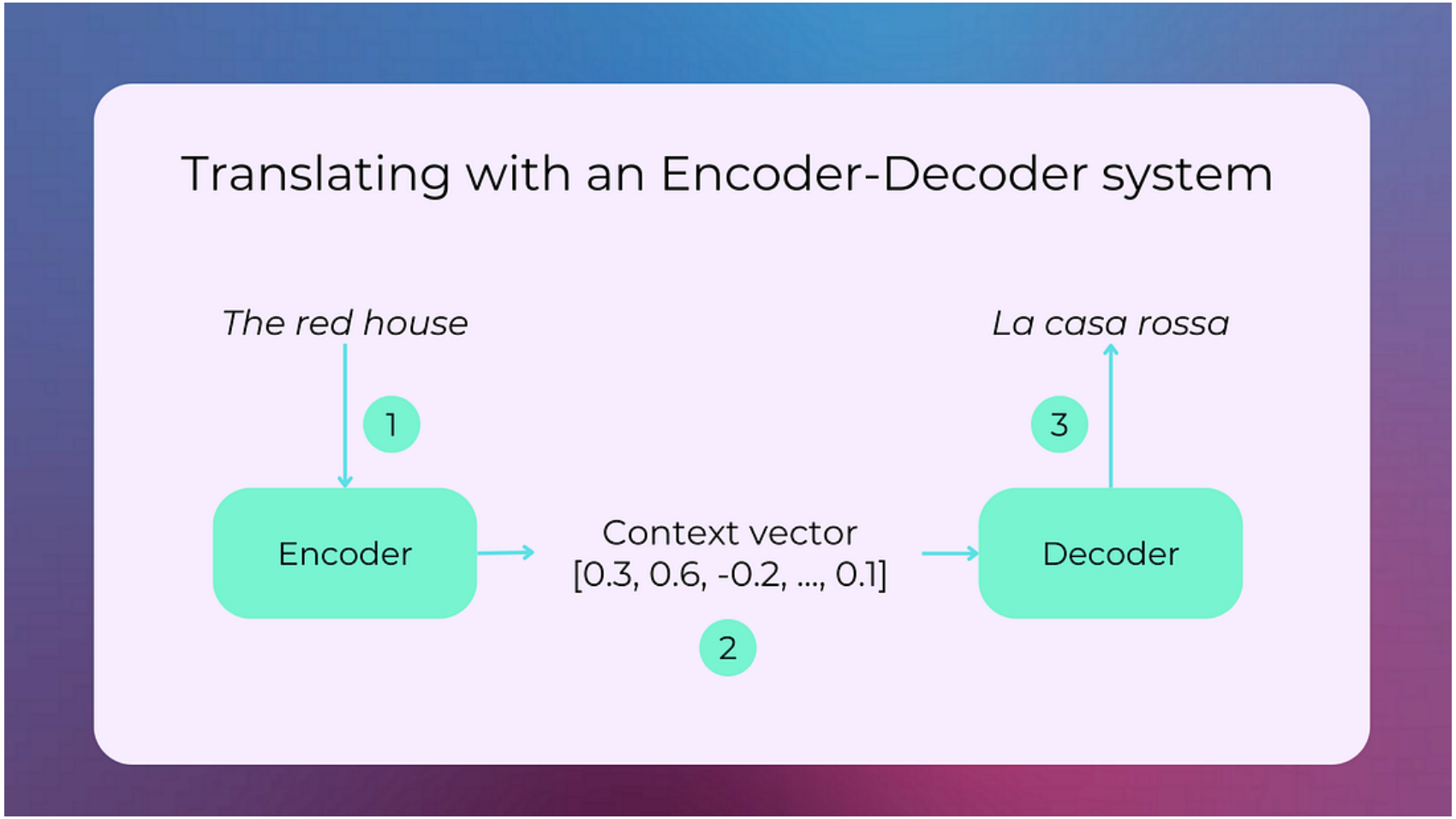
The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers

- What made word embeddings of great importance in NLP until today has been the evidence that using pre-trained embeddings as features for machine learning models improves performance across a wide range of downstream tasks.
- Since then, a lot of work has been done to obtain word embeddings that better encapsulate the meaning of the texts they represent.
- The most used neural networks during this period were LSTM RNNs and Convolutional Neural Networks (CNNs).

The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers

- In 2014, a general formalization of sequence-to-sequence problems (such as machine translation, which translates a sequence of words into another sequence of words) was proposed with a system made up of two neural networks called Encoder-Decoder: the first network is called Encoder and encodes the entire input into a vector (commonly called context vector), the second is called Decoder and decodes the output from the vector.

The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers



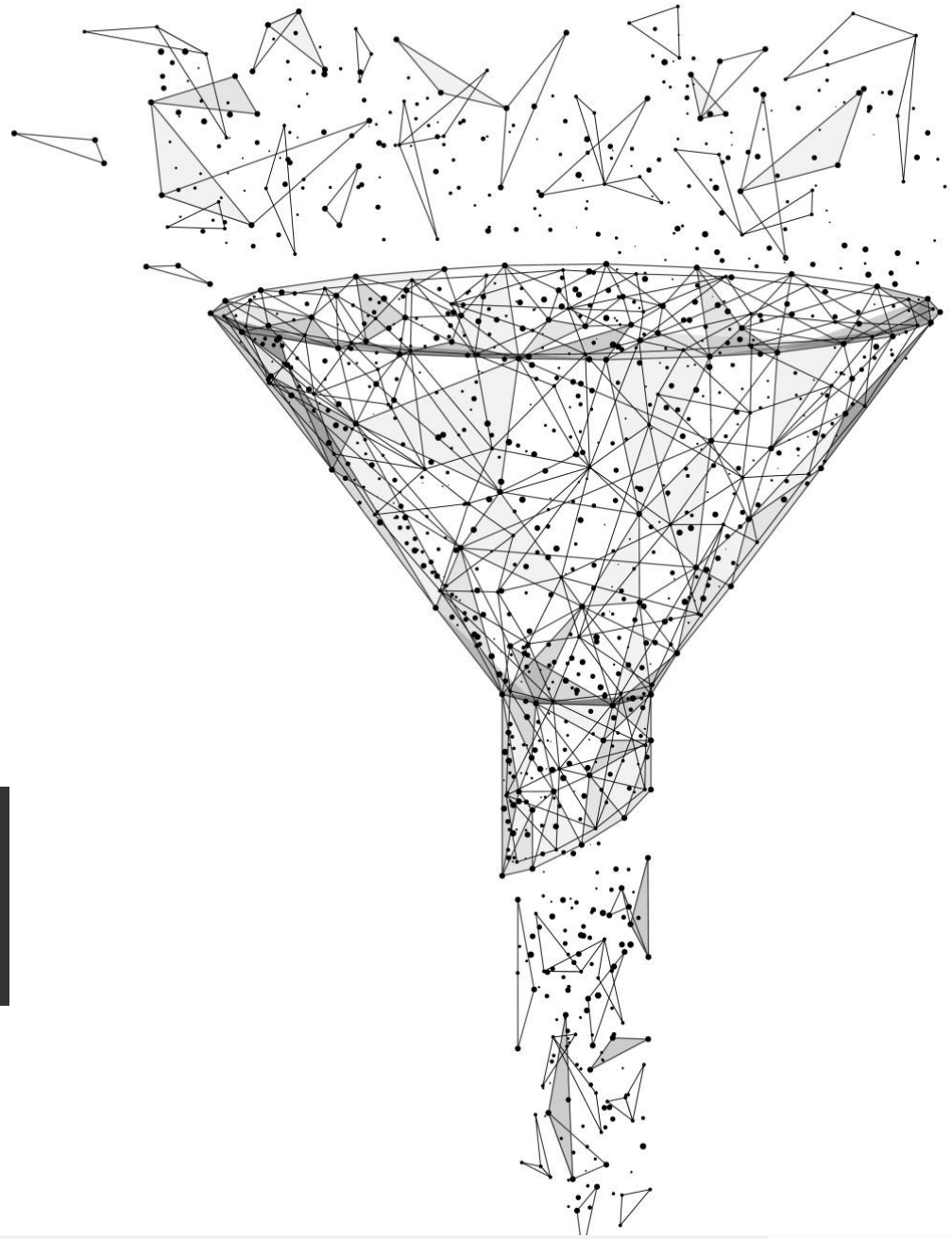
The 2000s, 2010s, and 2020s: Neural Networks, Embeddings, and Transformers

- The encoder-decoder architecture was then very successful with the use of attention mechanisms, which allow a greater passage of information between the two modules.
- This new system was so successful that, in 2016, Google Translate replaced its statistical model with a neural sequence-to-sequence model.
- The attention mechanism was then used in 2017 also to create a new type of neural network called Transformer (with the famous paper Attention Is All You Need), which is still the most used neural network in NLP today.
- Transformers are essentially language models trained in a self-supervised way on datasets comprising large quantities of texts from around the Internet.

References

- A Brief Timeline of NLP. A journey across grammars, expert… | by Fabio Chiusano | Generative AI | Medium
- From Rule-Based Systems to Transformers: A Journey through the Evolution of Natural Language Processing | by Omid (Saeid) Masoumzadeh, PhD | Medium

From Words to Vector



How Computer Understand Words?

- Computers cannot process (understand) words directly --> they must be numbers.
- How to convert words into numbers?
  - One-hot encoding → initial representation but the matrix is too sparse
  - Bag-of-words (BoW)
  - TF-IDF
  - n-gram
  - Word Embedding (Word2Vec, GloVe, FastText) → compressing meaning in vector space.

Word Embedding

- Word embeddings are **dense vector representations** of words, where similar words have similar vector values. For instance, the words “king” and “queen” would be close to each other in this vector space, reflecting their semantic similarity.
- Example:

Word	Example of 3D Vector Representation
Cat	[0.21, 0.83, 0.44]
Dog	[0.22, 0.79, 0.40]
Table	[0.87, 0.15, 0.02]

- The model understands that:
  - “cat” and “dog” are similar because their vectors are close in vector space (semantic similarity).
  - “table” is far away because their meanings are different.

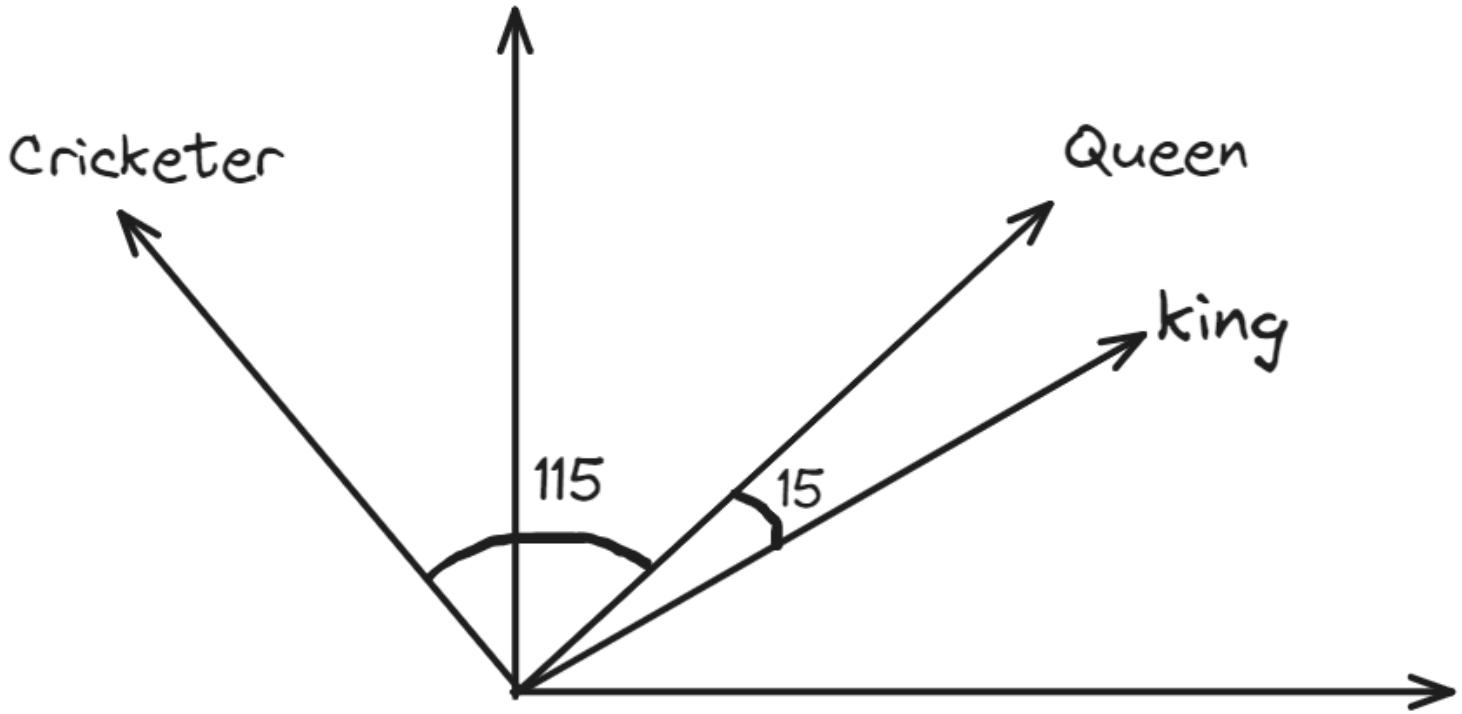
Word Embedding

- To further illustrate the concept of word embeddings, consider a simple example where words are passed into a neural network, and the network learns to represent each word as a 5-dimensional vector with random initial values.
- For instance, let's say our neural network assigns the following initial vectors to:
  - **King:** [0.1, 0.4, 0.8, 0.2, 0.9]
  - **Queen:** [0.3, 0.5, 0.6, 0.7, 0.8]
  - **Cricketer:** [0.9, 0.2, 0.1, 0.7, 0.5]
- As the neural network gets trained on a large corpus of text data, it adjusts these vectors to better represent the words' meanings and contexts.
- Through this learning process, the vectors of words with similar meanings or contexts become closer to each other in the 5-dimensional vector space, while those with dissimilar meanings or contexts move farther apart.

Word Embedding

- In this example, the final vectors learned by the neural network might look like:
  - **King:** [0.2, 0.3, 0.9, 0.5, 0.1]
  - **Queen:** [0.2, 0.4, 0.9, 0.5, 0.1]
  - **Cricketer:** [0.9, 0.4, 0.2, 0.1, 0.8]
- As we can see, the vectors for “king” and “queen” are now more similar, representing their semantic and contextual relationships (both being royalty).
  - Since the attributes of “king” and “queen” share similarities, their vector representations would be closer together in a graphical representation, indicating a smaller angle between them.
- On the other hand, the vector for “cricketer” remains distinct, highlighting its unrelated meaning compared to “king” and “queen”.
  - The attributes of “cricketer” are distinct from those of royalty, resulting in a larger angle between the “cricketer” vector and both “king” and “queen” vectors.

Word Embedding



Word Embedding

- Word2Vec vs GloVe: Which Word Embedding Model is Right for You? | by Amit Yadav | Biased-Algorithms | Medium
- (7) Word Embedding and Word2Vec, Clearly Explained!!! – YouTube
- GloVe: Global Vector - Introduction | Glove Word - Word Co-Occurrence Matrix | NLP | #18

RNN vs Transformer

RNN & LSTM

- What is LSTM (Long Short Term Memory)?
- (7) What is Recurrent Neural Network (RNN)? Deep Learning Tutorial 33 (Tensorflow, Keras & Python) - YouTube

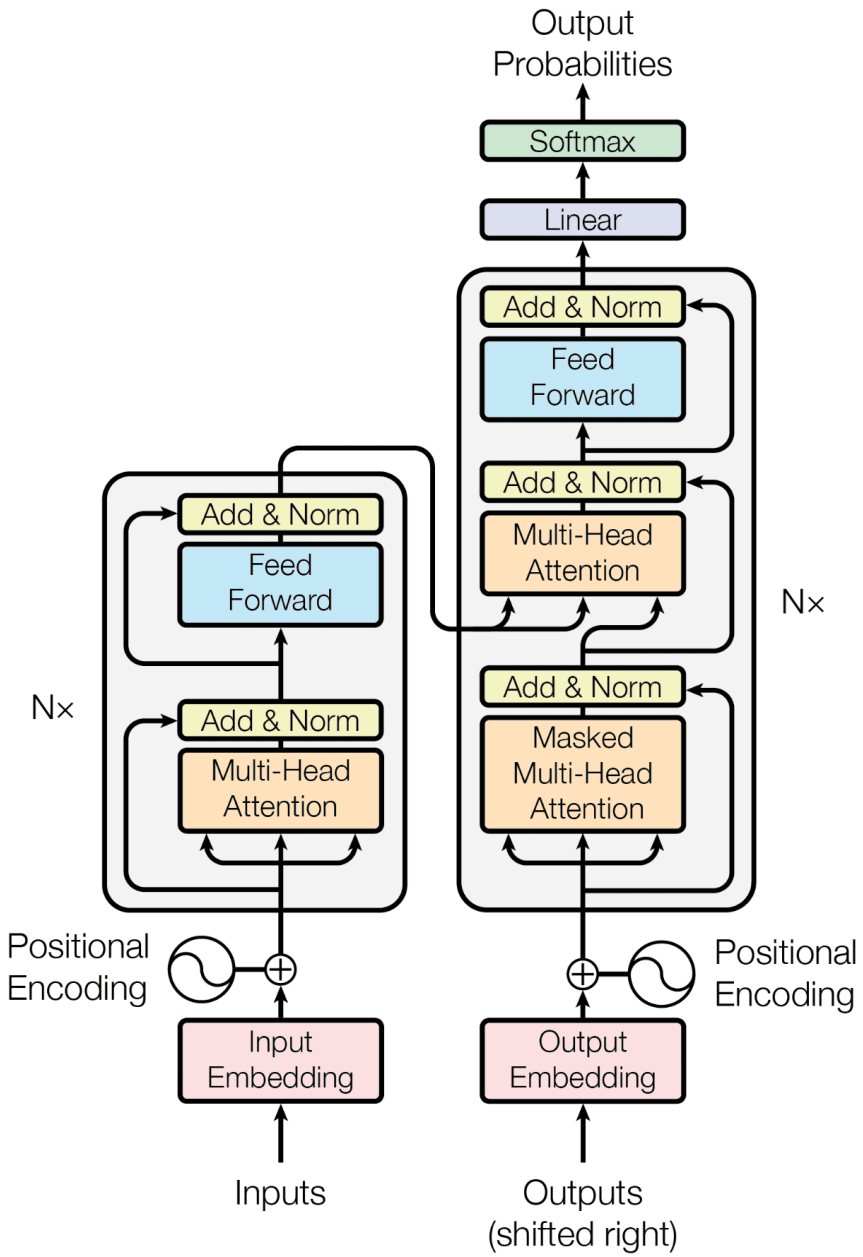
RNN (LSTM) vs Transformer

- Models like RNNs and LSTMs process data sequentially, one token after the other.
- This makes parallel processing difficult, and due to long paths, gradient problems (vanishing/exploding) and difficulties maintaining long contexts arise.
- Parallel processing means that the model can process all elements in a sentence simultaneously, not one by one.
- For example, the sentence: "Students are currently learning NLP.“
  - Old model (RNN/LSTM): Processes the word "Student" first, then "currently," then “learning," etc.
    - This is called sequential because the order must be followed from start to finish.
    - Consequence: slow and difficult to parallelize on a GPU.
  - Transformer Model: All words are seen simultaneously by the model.
    - The relationship between words is determined through the Attention mechanism, not through chronological order.
    - This means each token can be processed in parallel (simultaneously) on the GPU/TPU.

Why Parallel Processing is Needed?

- Speed and Efficiency
  - In RNNs, for a 100-word sentence, you have to wait 100 steps (1 per word).
  - Transformers can calculate attention weights for all words in one large step, making them much faster.
- Ability to See Deep Context
  - Because all tokens are seen simultaneously, Transformers can capture relationships between words at a distance, for example between "learning" and "NLP.“
  - RNNs tend to forget deep context (the vanishing gradient problem).
- Scalability
  - Because the process can be run in parallel on GPUs, Transformers can be trained on very large datasets (hundreds of billions of tokens).
  - This is what makes Large Language Models (LLMs) like GPT possible.

Transformer



Transformer

- [1706.03762] Attention Is All You Need
- Transformer Architecture Explained 'Attention Is All You Need'
- Attention is all you need explained
- Illustrated Guide to Transformers Neural Network: A step by step explanation

Problem with Word Embedding

1. An apple a day keeps a doctor away.
2. Apple is healthy.
3. Apple is better than orange.
4. Apple makes great phone.

Problem with Word Embedding

- We have four sentences, and we're using a two-dimensional embedding for the word "apple."
- In this embedding, the first dimension represents the taste of the apple, while the second dimension represents its association with technology.
- When we input each sentence into the neural network to generate the word embedding for "apple," the resulting outputs are represented in the following diagram.

Problem with Word Embedding

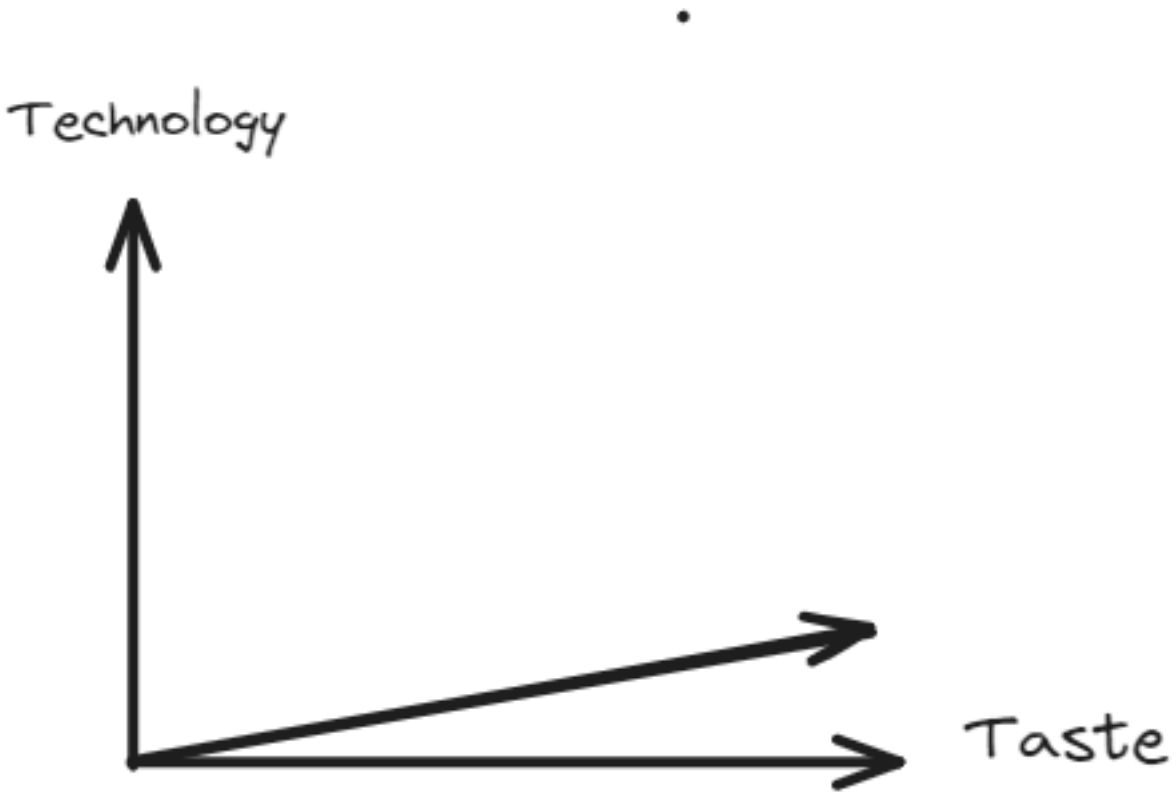
apple = [taste, technology]

1. An apple a day keeps a doctor away. [0.6, 0]
2. Apple is healthy. [0.7, 0]
3. Apple is better than orange. [0.8, 0]
4. Apple makes great phone. [0.75, 0.2]

Problem with Word Embedding

- Now, suppose our dataset consists of 10,000 sentences, with 9,000 referring to “apple” as a fruit and 1,000 mentioning it as a technology.
- As a result, the overall embedding vector for “apple” is [0.9, 0.3].
- This indicates that the average value of the apple embedding is skewed towards the taste dimension, reflecting the prevalence of sentences where “apple” is associated with its fruity context.
- The issue lies in the static nature of word embeddings.
- These embeddings are created once and remain unchanged, regardless of the varying contexts in which words are used across different datasets or tasks.
- This static representation fails to capture the dynamic nature of language and may not accurately reflect the nuances and variations in word meanings across different contexts.

Problem with Word Embedding



Problem with Word Embedding

- Consider the scenario of building an English to Hindi translator.

"Apple launched a new iPhone while I was eating an orange"

- Initially, we generate word embeddings for each word in the input sentence.
- For instance, if the embedding for "apple" is [0.9, 0.3], it suggests a strong association with the fruit due to its higher value in the taste dimension.
- However, in the given sentence where "apple" refers to a phone, this static embedding fails to capture the context accurately.

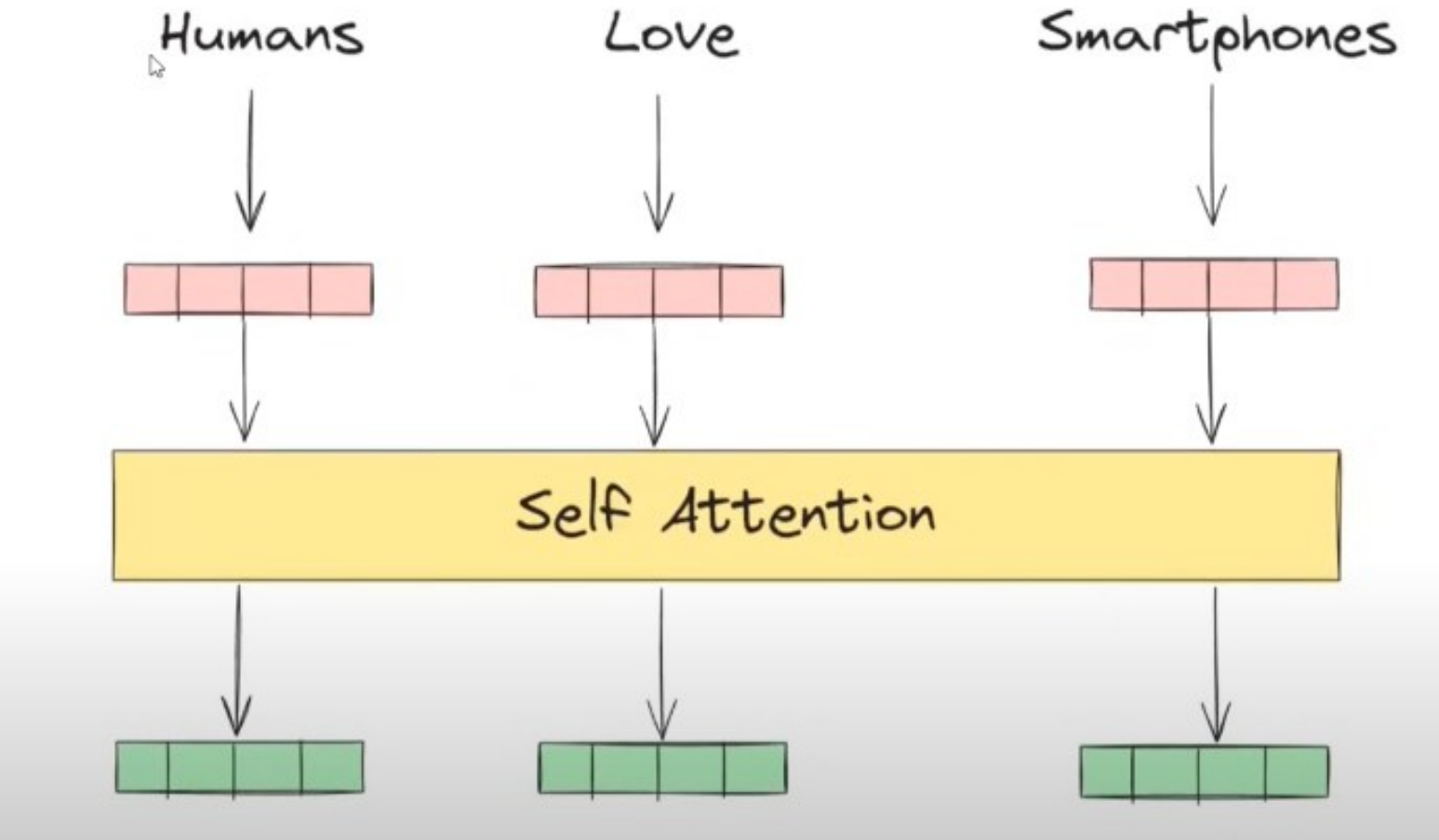
Problem with Word Embedding

- To address this limitation, we need contextual embeddings that adapt based on the surrounding words in the sentence.
- Self-attention offers a solution by dynamically adjusting embeddings according to the context in which words are used.
- In the example sentence, when words like “launched” and “phone” appear, the embedding for “apple” would adjust, reducing its association with taste and increasing its association with technology.
- This contextual awareness prevents confusion, even when words like “orange” are present.

Self-attention in Transformer

- By passing word embeddings through self-attention mechanisms, we obtain smart contextual embeddings that accurately reflect the meaning of words in their specific contexts, improving the performance of NLP tasks like translation.
- From above discussion, it's evident that self-attention serves as a mechanism for transforming static embeddings into dynamic contextual embeddings.
- See the diagram below: Beginning with the sentence 'Humans love smartphones,' each word undergoes static embedding generation.
- These embeddings are then fed into a self-attention block, where dynamic contextual embeddings are produced through the mechanism of capturing inter-word dependencies.

Self-attention in Transformer



Self-attention in Transformer

- Self-attention is a mechanism where each token (word or piece) in an input sequence attends to *all* the other tokens (including itself) to build a context-aware representation.
  - For each token we compute **Query (Q)**, **Key (K)** and **Value (V)** vectors derived from the token embedding
  - We compute attention weights using the dot-product of query with keys, scale by √(d_k) (the dimension of the key vectors), apply softmax, then multiply by the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

  - The resulting output for each token is a weighted sum of the value vectors of all tokens, so each output vector integrates global contextual information from the full sequence.
- Key benefits: allows capturing long-range dependencies, enables parallelism (since all tokens see all others simultaneously), and replaces recurrent/ sequential structure.

Multi-head Attention in Transformer

- Multi-head attention builds on self-attention by running **several self-attention mechanisms (heads) in parallel**, each with its own projection of Q, K, V. Then the outputs of all these heads are concatenated and linearly transformed.
- Each head allows the model to learn different types of relationships or "perspectives" in the data, for example one head might focus on syntactic relations, another on semantic similarity.
- Mathematically:

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V) \quad \text{for } i = 1, \ldots, h$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\ W^O$$

where W_i^Q, W_i^K, W_i^V are learnable matrices for head i, and W^O projects the concatenation back to model dimension.

Multi-head Attention in Transformer

- Advantages:
  - Captures richer, more varied contextual relationships.
  - Improves model robustness by not relying on a single “attention pattern”.
  - Maintains parallel processing benefits of self-attention while expanding expressiveness.
- In the Transformer architecture, multi-head attention appears in encoder self-attention, decoder self-attention (often masked), and encoder-decoder cross-attention layers.

Positional Encoding in Transformer

- By using self-attention and multi-head attention, transformers excel at capturing relationships between words by focusing on different parts of a sequence.
- However, these mechanisms alone don't tell the model where each word is in a sentence.
- Since word order is crucial for understanding meaning, transformers need a way to know the position of each word.
- This is where positional encoding comes in.
- It adds information about the order of words, allowing the model to make sense of sequences like sentences.

Positional Encoding in Transformer

- Each position in a sequence (1st, 2nd, 3rd, etc.) gets a unique **positional vector** that is **added** to the token embedding before feeding into the Transformer.
- Mathematically:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- where:
  - pos= position of the token in the sequence
  - i= dimension index
  - d_{model} = embedding size (e.g. 512)
- So each token's final input is:

$$\text{Input vector} = \text{Token embedding} + \text{Positional encoding}$$

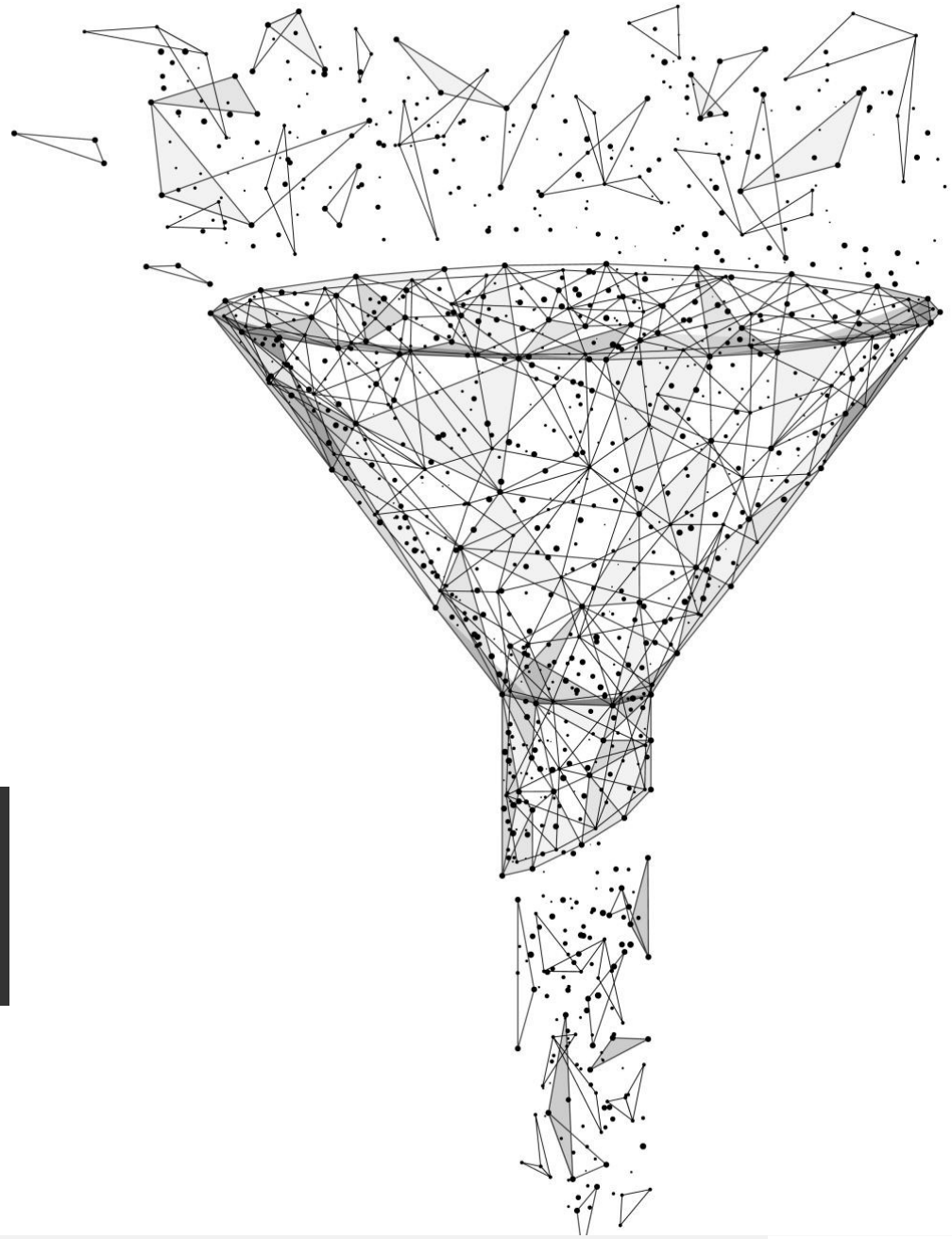
Transformer

- With attention and block architecture (multi-head attention + feed-forward + residuals + norm layer), very large models (thousands of parameters) can be built and trained on large datasets more efficiently.
- This paves the way for large language models (LLMs).
- Because LLM requires a broad understanding of context and the ability to generalize from big data, the Transformer foundation is a natural fit.
- Models like BERT, GPT-3, and T5 are all built on the Transformer architecture (or variations thereof).

References

- The Illustrated Transformer – Jay Alammar – Visualizing machine learning one concept at a time.
- Understanding Self Attention in Transformers | by Sachinsoni | Medium
- Positional Encoding in Transformer | by Sachinsoni | Medium

Transformer to LLM



Transformer to LLM

- The basic architecture of Large Language Models (LLMs) is rooted in the Transformer.
- These large models utilize numerous parameters (billions or even trillions), very large datasets (tens to hundreds of billions of tokens), and very high computational power.
- Key concepts: pre-training and fine-tuning.
  - Pre-training: The model is trained with a general objective (e.g., next-word prediction or masked word prediction) on large data, without a specific task.
  - Fine-tuning: The model is then adapted to a specific task (e.g., Q&A, summarization) with a smaller dataset.
- Architecture types:
  - Encoder-only (e.g., BERT): Good for representation and classification tasks.
  - Decoder-only (e.g., GPT series): Generates text auto-regressively.
  - Encoder-decoder (e.g., T5): Converts all tasks to a “text-to-text” format.

Transformer Architectures

- BERT (Bidirectional Encoder Representations from Transformers): An encoder-only model that uses Masked Language Modeling (MLM) to learn bidirectional context.
  - Powerful for tasks like classification, NER, and QA (non-generative).
- GPT (Generative Pre-trained Transformer): Decoder-only, trained with Causal Language Modeling (next-token prediction) in an auto-regressive manner.
  - Suitable for text generation.
- T5 (Text-to-Text Transfer Transformer): An encoder-decoder architecture that equates all tasks to a "text input → text output" format.
  - Flexible for many tasks.

Transformer Architectures

- Transformers, explained: Understand the model behind GPT, BERT, and T5
- The Evolution of Large Language Models (LLMs) | WordGPT Blog

Transformer Architectures

- Pros & Cons:
  - BERT: Good for comprehension, poor generation.
  - GPT: Good generation, requires fine-tuning to be relevant to specific tasks and avoid hallucinations.
  - T5: Very flexible, slightly more complex in architecture and training requirements.
- Understanding model types helps choose the right architecture for their problem (e.g. chatbot, summarization, classification).

NLP → LLM

- LLM (Large Language Model) is the latest evolution of the field of NLP (Natural Language Processing)
  - Large-scale deep learning (transformer-based)
  - Using trillions of tokens from the internet, books, articles, etc.
    - NLP usually using domain-specific token
  - One model can perform multiple tasks via prompts.
  - Interact with users directly using natural language ("prompts")
- Example of LLM:
  - GPT-4 / GPT-5 (OpenAI)
  - Gemini (Google)
  - Claude (Anthropic)
  - LLaMA (Meta)
  - Mistral, Falcon, dll.

Advancements in LLMs

1. Scaling Laws and Model Size
2. Pre-training and Fine-tuning Paradigms
3. Emergence of Multimodal Models

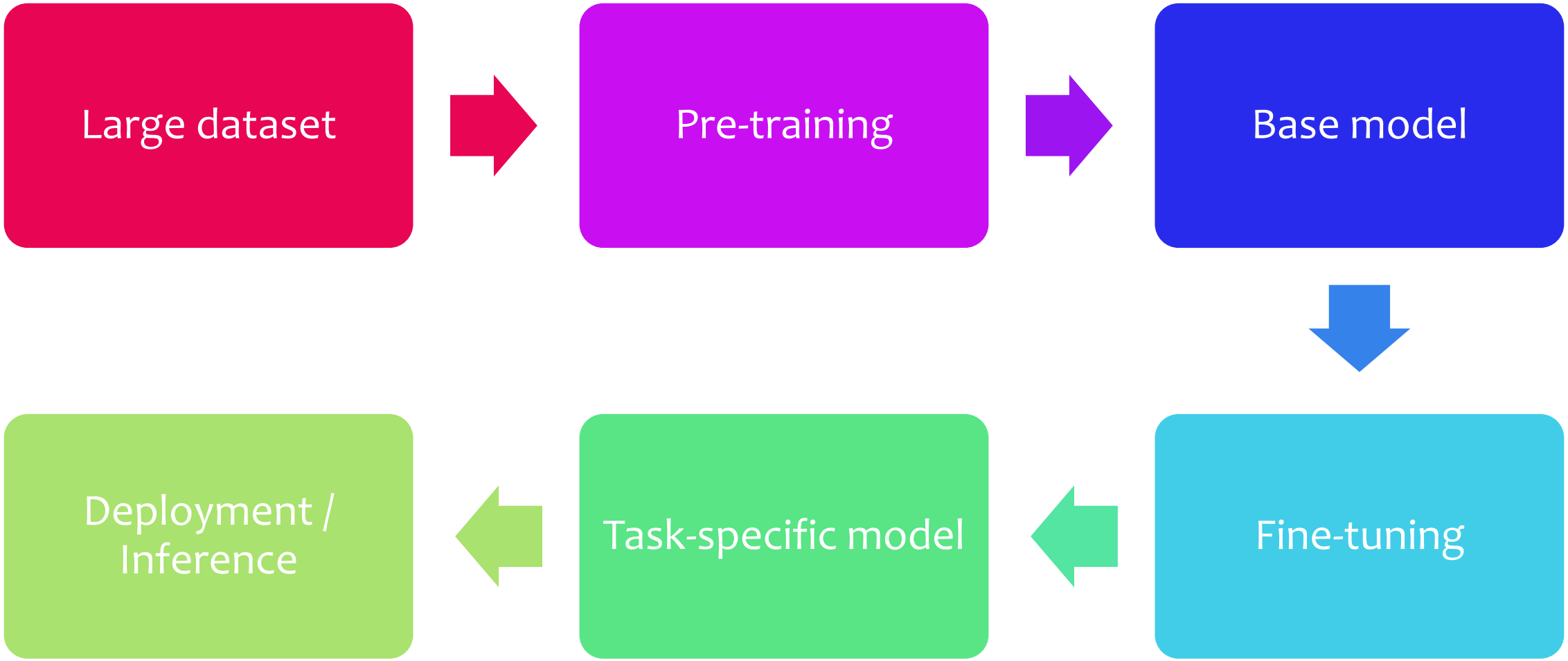
Scaling Laws & Model Size

- "Scaling laws" in the context of LLM refer to the empirical relationship between model size (number of parameters), number of tokens in the training data, and compute (number of FLOPs operations) with model performance.
- Key finding: Model performance predictably increases **in a power-law manner** as parameters, data, and compute are increased; but with diminishing returns after a certain point.
- Implication: To achieve a highly powerful LLM, it's not just the size that matters, but the balance between parameters/data/compute.
- Training and inference costs are substantial; this gives rise to techniques like distillation, quantization, and mixture of experts for scale-effectiveness.
- References:
  - https://arxiv.org/abs/2001.08361
  - https://www.youtube.com/watch?v=FUqzdvlaoRk&pp=ygUPc2NhbGluZyBsYXcgbGxt

Pre-training & Fine-tuning Paradigms

- **Pre-training**: The model is first trained with a general objective (e.g., next-word prediction or masked words) on large, unlabeled data. The goal is for the model to learn a broad language representation.
- **Fine-tuning**: After pre-training is complete, the model is adapted to a specific task with a smaller, domain-specific dataset. This allows for adaptation to real-world problems (e.g., campus chatbots, departmental QA systems, MSME document analysis).
- **Inference/Deployment**: Once the model is ready, it is deployed in a real-world application: generating text, answering questions, summarizing, etc. At this stage, issues such as latency, cost, quantization, and batch vs. real-time inference arise.

Pre-training & Fine-tuning Paradigms



Pre-training & Fine-tuning Paradigms

- The pre-training and fine-tuning paradigm became a standard approach in LLM development.
- Pre-training on vast corpora allowed models to learn general language representations, which could be fine-tuned for specific tasks.
- This approach improved performance and reduced the need for extensive task-specific data.

Emergence of Multimodal Models

- The emergence of multimodal models extended LLM capabilities beyond text to include images and other modalities.
- Models like DALL-E and CLIP demonstrated the potential for cross-modal understanding and generation, enabling applications in art, design, and content creation.
- These models combined text and visual data, offering new avenues for AI research and innovation.

Thank You